Monad RPC

Security Assessment

March 20th, 2026 — Prepared by OtterSec

James Wang james.wang@osec.io

Jinwoo Lee jin@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-MRC-ADV-00 | Uncapped WebSocket Notification Amplification 6

OS-MRC-ADV-01 | Delayed Response Size Enforcement 7

OS-MRC-ADV-02 | Excessive Default Batch Limit 8

General Findings **9**

OS-MRC-SUG-00 | Acceptance of Trailing RLP Bytes 10

OS-MRC-SUG-01 | Excessive Request Body Logging at Info Level 11

Vulnerability Rating Scale **12**

Procedure **13**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Category Labs engaged OtterSec to assess the `rpc` program. This assessment was conducted between January 2nd and January 9th, 2026. For more information on our auditing methodology, refer to chapter 07.

Key Findings

We produced 5 findings throughout this audit engagement.

In particular, we identified a vulnerability where WebSocket notifications are not size-capped and the per-connection subscription limit is high, allowing a single client to amplify large log streams into excessive outbound traffic (OS-MRC-ADV-00). Additionally, the maximum response size check is enforced only after a response is fully generated, allowing large requests to consume significant memory prior to rejection (OS-MRC-ADV-01). Furthermore, the current batch limit of 5000 allows a single request to trigger a large amount of RPC work, particularly for expensive or large-response methods, potentially consuming significant resources (OS-MRC-ADV-02).

We also recommend enforcing full input consumption after decoding, as the current logic may accept raw transaction payloads with trailing RLP bytes (OS-MRC-SUG-00). Moreover, we advise avoiding logging full request bodies at the info level, as this can generate excessive log volume and increase resource usage in production environments (OS-MRC-SUG-01).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a git repository at https://github.com/category-labs/monad. This audit was performed against commit 5130f0e.

A brief description of the programs is as follows:

Name	Description
rpc	The client-facing API server for a Monad node, exposing Ethereum-compatible and Monad-specific JSON-RPC and optional WebSocket interfaces.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 5 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.

Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	3
INFO	2

© 2026 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in chapter 06.

ID	Severity	Status	Description
OS-MRC-ADV-00	LOW	RESOLVED	WebSocket notifications are not size-capped, and the per-connection subscription limit is high, allowing one client to amplify large log streams into very large outbound traffic.
OS-MRC-ADV-01	LOW	RESOLVED	`max_response_size` is enforced only after the response is fully generated, enabling large requests to still consume significant memory before they are rejected.
OS-MRC-ADV-02	LOW	RESOLVED	A batch limit of 5000 allows a single request to trigger a very large amount of RPC work, especially for expensive or large-response methods, which may consume substantial amount of resources.

© 2026 Otter Audits LLC. All Rights Reserved.

Uncapped WebSocket Notification Amplification LOW OS-MRC-ADV-00

Description

The WebSocket path does not enforce a response-size cap and allows up to 100 subscriptions per connection. For high-volume streams such as `MonadLogs`, a single client can replicate large per-block log payloads across multiple subscriptions on the same connection.

`MonadLogs` emits approximately four notifications per block, and a single block may produce on the order of 10 MB of logs (e.g., under a 200M gas limit). This can result in up to 4 GB of data transmitted per block over a single WebSocket connection. As such, this behavior introduces a potential resource exhaustion vector.

Remediation

Impose tighter per-connection subscription limits and bound the size of individual notifications.

Patch

Resolved in PR#2733.

© 2026 Otter Audits LLC. All Rights Reserved.

Delayed Response Size Enforcement LOW OS-MRC-ADV-01

Description

In the current implementation, `max_response_size` is enforced only after the full RPC response has been computed, immediately prior to transmission. As a result, memory is already consumed before the limit is applied.

Large-output endpoints such as `debug_getRawBlock` and `debug_getRawReceipts` may allocate substantial amounts of data, potentially constructing responses up to the maximum batch size before enforcement. This reduces the effectiveness of the limit as a safeguard against excessive memory usage.

Remediation

Enforce the response size limit check earlier in the execution flow to effectively prevent excessive resource consumption.

Patch

Resolved in PR#2959.

© 2026 Otter Audits LLC. All Rights Reserved.

Excessive Default Batch Limit LOW OS-MRC-ADV-02

Description

The default batch limit of 5000 allows a single request to trigger a large number of RPC method executions, concentrating resource usage within a single request. This is particularly problematic when batched methods involve expensive operations or return large payloads.

As a result, the current limit may be overly permissive for deployments that expose high-cost or high-volume endpoints.

Remediation

Lower the default batch limit.

Patch

Resolved in PR#2934.

© 2026 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MRC-SUG-00	`sendRawTransaction` may accept raw transaction payloads with trailing RLP bytes because it does not enforce full input consumption after decoding.
OS-MRC-SUG-01	The code logs full RPC request contents at the info level, resulting in unnecessarily large log volumes and increased resource usage.

© 2026 Otter Audits LLC. All Rights Reserved.

Acceptance of Trailing RLP Bytes

OS-MRC-SUG-00

Description

`sendRawTransaction` decodes the submitted bytes into a transaction but does not verify that the entire input is consumed, allowing a valid transaction with trailing RLP data to be accepted. However, since the extra bytes are ignored and only the decoded transaction is propagated and executed, this behavior does not appear to introduce a practical issue.

Remediation

Reject any input for which unread bytes remain after decoding.

Patch

Resolved in PR#2802.

© 2026 Otter Audits LLC. All Rights Reserved.

Excessive Request Body Logging at Info Level OS-MRC-SUG-01

Description

Logging full request bodies or parameters at the `info` level can generate excessive log volume, especially for RPC methods with large inputs or responses. Currently, both `info!("monad_debug_traceCall: params:?")` in `monad_debug_traceCall` and `info!("response exceed size limit: body:?")` in `rpc_handler` may log entire RPC payloads.

Since `info` logs are typically enabled in production and often exported to centralized logging systems, this behavior can significantly increase bandwidth, storage usage, and log processing overhead. Under high request throughput, this may degrade system performance and increase operational costs.

Remediation

Avoid logging full request bodies at the info level.

Patch

Resolved in PR#2737.

© 2026 Otter Audits LLC. All Rights Reserved.

06 — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

07 — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.